Модуль 5

Express.js — основы

Установка • Роутинг • Параметры • Middleware • Статика • Обработка ошибок • Router

5.1 Зачем нужен Express.js

В предыдущем модуле мы создали HTTP-сервер на чистом Node.js. Это работает, но для реального проекта неудобно: приходится вручную парсить URL, тело запроса, реализовывать роутинг через if/else. Express.js решает все эти проблемы.

Express.js — самый популярный веб-фреймворк для Node.js. Он предоставляет:

- Удобный роутинг с поддержкой параметров (/users/:id)
- Систему middleware — функции, которые обрабатывают запрос по цепочке
- Автоматический парсинг JSON, форм, query-параметров
- Отдачу статических файлов одной строкой
- Модульную организацию роутов (Router)
- Централизованную обработку ошибок

Express минималистичен — он не навязывает структуру проекта и не включает ORM, шаблонизатор или систему аутентификации. Вы сами выбираете нужные инструменты.

Установка:

```
npm init -y
npm install express
```

Express работает как надстройка над встроённым модулем http в Node.js. Под капотом Express создаёт обычный HTTP-сервер, но оборачивает его в удобные абстракции: объект приложения (app), маршруты (routes), цепочку middleware и методы ответа (res.json, res.status, res.send и т.д.). Благодаря этому разработчик пишет значительно меньше шаблонного кода.

Создаём первый сервер на Express

Создайте папку проекта и инициализируйте npm:

```
mkdir my-express-app
cd my-express-app
npm init -y
```

Переключите проект на ESM-модули.

Откройте файл package.json и добавьте в него поле "type": "module". Это позволит использовать синтаксис import/export вместо require/module.exports. Без этой настройки Node.js будет считать файлы .js файлами CommonJS и выдаст ошибку при использовании import.

Пример package.json после изменения:

```
{
  "name": "my-express-app",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "start": "node server.js"
  }
}
```

Установите фреймворк Express:

```
npm install express
```

Создайте файл server.js и добавьте в него следующий код:

```
// server.js
import express from 'express';

const app = express();
const PORT = process.env.PORT || 3000;

// Middleware для парсинга JSON в теле запроса
app.use(express.json());

// Определяем маршрут
app.get('/', (req, res) => {
    res.json({ message: 'Привет, Express!' });
});

// Запускаем сервер
app.listen(PORT, () => {
    console.log(`Сервер: http://localhost:${PORT}`);
});
```

Запустите проект

```
node server.js
```

затем откройте браузер http://localhost:3000 и вы должны увидеть

Привет, Express!

Это значит, что все работает и вы запустили свой первый сервер на Express.js

Остановку сервера можно выполнить нажатием CTRL + C.

В сравнении с чистым Node.js: не нужно вручную парсить URL, устанавливать Content-Type (res.json() делает это автоматически), писать res.end(). Код стал значительно короче и читабельнее.

5.2 Роутинг

Роутинг в Express определяется через методы app.get(), app.post(), app.put(), app.patch(), app.delete(). Каждый метод принимает путь и callback-функцию (обработчик).

Что такое роутинг?

Роутинг (routing) — это механизм, который определяет, какой код должен выполниться при обращении клиента к определённому URL-адресу определённым HTTP-методом. Например, GET-запрос на /about и POST-запрос на /api/users — это два разных маршрута, и каждый обрабатывается своей функцией.

В Express маршрут определяется тремя частями:

1) HTTP-метод — GET, POST, PUT, PATCH, DELETE и т.д.

2) Путь (path) — строка URL, например / или /api/users/:id.

3) Обработчик (handler) — callback-функция, которая получает объекты req (запрос) и res (ответ).

Добавляем маршруты в сервер

Откройте файл server.js, который мы создали в предыдущем разделе.

Добавьте простой GET-маршрут:

```
app.get('/', (req, res) => {
    res.json({ message: 'Главная страница' });
});
```

Когда пользователь откроет http://localhost:3000/ в браузере (это GET-запрос), Express вызовет именно эту функцию.

Добавьте маршрут с параметром:

```
app.get('/users/:id', (req, res) => {
    const userId = req.params.id;
    res.json({ message: `Запрошен пользователь с ID: ${userId}` });
});
```

Двоеточие перед id означает, что это динамический параметр. При запросе GET /users/42 значение req.params.id будет равно строке "42".

Добавьте POST-маршрут:

```
app.post('/users', (req, res) => {
    const { name, email } = req.body;
    res.status(201).json({ message: 'Пользователь создан', name, email
});
});
```

Для POST-запросов данные приходят в req.body. Чтобы Express автоматически парсил JSON из тела запроса, убедитесь, что у вас подключён middleware: app.use(express.json());

Запустите сервер и проверьте маршруты:

```
node server.js
```

Проверка через curl:

```
curl http://localhost:3000/
curl http://localhost:3000/users/42
curl -X POST http://localhost:3000/users -H 'Content-Type:
application/json' -d '{"name":"Иван","email":"ivan@test.com"}'
```

Обратите внимание: порядок маршрутов имеет значение. Express проверяет маршруты сверху вниз и выполняет первый подходящий. Если у вас есть маршрут /users/:id и /users/admin, маршрут /users/admin нужно объявить первым, иначе Express посчитает "admin" параметров :id.

Теперь рассмотрим объекты req и res подробнее.

Объект req (запрос) содержит:

- req.params — параметры из URL (req.params.id для /users/:id)
- req.query — query-параметры (?page=2&limit=10)
- req.body — тело запроса (для POST, PUT, PATCH)
- req.headers — заголовки запроса
- req.method — HTTP-метод (GET, POST...)

Объект res (ответ) содержит:

- res.json(data) — отправить JSON-ответ (автоматически устанавливает Content-Type)
- res.status(code) — установить статус-код (можно цепочкой: res.status(201).json(data))
- res.send(data) — отправить ответ (строку, Buffer, объект)
- res.set(header, value) — установить заголовок ответа
- res.end() — завершить ответ без тела

Пример полного CRUD API:

```
let users = [
    { id: 1, name: 'Алексей', email: 'alex@example.com' },
    { id: 2, name: 'Мария', email: 'maria@example.com' },
];

// GET — получить всех пользователей
app.get('/api/users', (req, res) => {
    res.json(users);
});
```

```
// GET — получить одного по ID (параметр :id)
app.get('/api/users/:id', (req, res) => {
    const user = users.find(u => u.id === +req.params.id);
    if (!user) return res.status(404).json({ error: 'Не найден' });
    res.json(user);
});

// POST — создать нового
app.post('/api/users', (req, res) => {
    const { name, email } = req.body;
    const user = { id: Date.now(), name, email };
    users.push(user);
    res.status(201).json(user);  // 201 = Created
});

// PATCH — частичное обновление
app.patch('/api/users/:id', (req, res) => {
    const user = users.find(u => u.id === +req.params.id);
    if (!user) return res.status(404).json({ error: 'Не найден' });
    Object.assign(user, req.body);  // Обновляем только переданные поля
    res.json(user);
});

// DELETE — удалить
app.delete('/api/users/:id', (req, res) => {
    const index = users.findIndex(u => u.id === +req.params.id);
    if (index === -1) return res.status(404).json({ error: 'Не найден'
});
    users.splice(index, 1);
    res.status(204).end();  // 204 = No Content
});
```

Обратите внимание на +req.params.id — параметры URL всегда строки, поэтому преобразуем в число оператором +.

5.3 Query-параметры

Query-параметры передаются в URL после ? и доступны через req.query. Express парсит их автоматически:

```
// GET /api/products?category=phones&sort=price&page=2
app.get('/api/products', (req, res) => {
    const { category, sort, page = 1, limit = 10 } = req.query;
    // page и limit имеют значения по умолчанию
    // Все значения — строки! Преобразуйте в числа при необходимости
    res.json({ products: [], page: +page, limit: +limit });
});
```

Что такое query-параметры?

Query-параметры — это пары ключ=значение, которые передаются в URL после символа ?. Несколько параметров разделяются символом &. Они используются для фильтрации, сортировки, пагинации и передачи дополнительных данных без изменения самого маршрута.

Пример URL с query-параметрами:

```
http://localhost:3000/api/products?category=phones&sort=price&order=asc
&page=2&limit=10
```

В этом URL:

category=phones — фильтр по категории

sort=price — сортировка по цене

order=asc — порядок сортировки (по возрастанию)

page=2 — номер страницы

limit=10 — количество элементов на страницу

Важно помнить: все значения query-параметров приходят как строки. Если вам нужно число — преобразуйте явно через Number(), parseInt() или оператор +.

Добавляем маршрут с query-параметрами

Откройте файл server.js.

Добавьте маршрут для списка продуктов с фильтрацией:

```
app.get('/api/products', (req, res) => {
    const { category, sort = 'name', order = 'asc', page = '1', limit =
'10' } = req.query;

    // Преобразуем строки в числа

    const pageNum = Math.max(1, +page);

    const limitNum = Math.max(1, Math.min(100, +limit));
```

```
    res.json({
        filters: { category, sort, order },
        pagination: { page: pageNum, limit: limitNum },
        products: [] // здесь будут данные из БД
    });
});
```

Запустите сервер и проверьте:

```
node server.js
```

Проверьте через curl или браузер:

```
curl 'http://localhost:3000/api/products'
curl
'http://localhost:3000/api/products?category=phones&sort=price&page=2'
```

Первый запрос вернёт значения по умолчанию (page=1, limit=10). Второй — параметры, которые мы передали в URL.

Разница между req.params и req.query:

req.params — это параметры из самого пути (/users/:id). Они обязательны: если не передать id, маршрут просто не сработает.

req.query — это параметры после знака ?. Они всегда необязательны: если не передать, значение будет undefined (поэтому задавайте значения по умолчанию).

5.4 Middleware — ключевая концепция Express

Middleware (промежуточное ПО) — это функции, которые выполняются в цепочке между получением запроса и отправкой ответа. Каждый middleware получает три аргумента: req, res и next. Вызов next() передаёт управление следующему middleware в цепочке.

Middleware может:

- Выполнить любой код (логирование, проверка авторизации)
- Изменить объекты req и res (добавить свойства, установить заголовки)
- Завершить цикл запрос-ответ (отправить ответ, не вызывая next)
- Передать управление следующему middleware (вызвать next())

```
// Пример: middleware для логирования
const logger = (req, res, next) => {
    const start = Date.now();
    res.on('finish', () => {
        const duration = Date.now() - start;
        console.log(`${req.method} ${req.url} ${res.statusCode}
${duration}ms`);
    });
    next(); // ОБЯЗАТЕЛЬНО вызвать next(), иначе запрос «зависнет»
};


// Применить ко ВСЕМ роутам
app.use(logger);
```

Порядок middleware важен! Они выполняются в том порядке, в котором зарегистрированы через app.use(). Поэтому логгер нужно регистрировать до роутов, а обработчик ошибок — после.

Встроенные middleware Express:

```
app.use(express.json());                          // Парсинг JSON-тела
app.use(express.urlencoded({ extended: true }));  // Парсинг HTML-форм
app.use(express.static('public'));                // Статические файлы
```

express.json() — парсит тело запроса с Content-Type: application/json и помещает результат в req.body. Без этого middleware req.body будет undefined.

express.static("public") — отдаёт файлы из папки public/. Запрос GET /style.css вернёт файл public/style.css. Одна строка заменяет весь код отдачи статики из учебного Модуля 4.

Что такое middleware простыми словами?

Представьте конвейер по сборке автомобилей на заводе: каждый автомобиль проходит через несколько сборщиков, и каждый из них может что-то сделать с этим автомобилем, прежде чем передать его дальше по конвейеру для последующей сборки. Это и есть middleware.

Еще одна простая аналогия: когда вы заходите в здание, вы проходите через охрану (проверка авторизации), затем ресепшен (логирование визита), затем попадаете в нужный кабинет (обработчик маршрута). Каждый этап — это middleware.

Сигнатура middleware-функции:

```
const myMiddleware = (req, res, next) => {
    // req — объект запроса
    // res — объект ответа
    // next — функция, которая передаёт управление следующему
middleware
    next();
};
```

Если middleware не вызывает next(), запрос «зависнет» — клиент не получит ответ, и через некоторое время произойдёт таймаут.

Создаём свой middleware

Откройте файл server.js.

Создайте middleware для логирования запросов. Добавьте этот код ДО объявления маршрутов:

```
const logger = (req, res, next) => {
    const start = Date.now();
    console.log(`--> ${req.method} ${req.url}`);

    res.on('finish', () => {
        const duration = Date.now() - start;
```

```
        console.log(`<-- ${req.method} ${req.url} ${res.statusCode}
${duration}ms`);
    });

    next();
};
```

Подключите middleware к приложению:

```
app.use(logger);
```

Метод app.use() без указания пути применяет middleware ко ВСЕМ маршрутам.

Запустите сервер и сделайте любой запрос:

```
node server.js
curl http://localhost:3000/
```

В консоли вы увидите лог вида:

--> GET /

<-- GET / 200 3ms

Создайте middleware для проверки авторизации:

```
const auth = (req, res, next) => {
    const token = req.headers['authorization'];
    if (!token) {
        return res.status(401).json({ error: 'Требуется авторизация'
});
    }
    // В реальном приложении здесь проверяется JWT-токен
    next();
};
```

Примените auth только к определённым маршрутам:

```
app.get('/api/profile', auth, (req, res) => {
    res.json({ user: 'Информация о пользователе' });
});
```

Здесь auth передаётся вторым аргументом — он выполнится только для маршрута /api/profile, а не для всех маршрутов.

Проверьте:

```
curl http://localhost:3000/api/profile
```

Ответ будет:

{ "error": "Требуется авторизация" } (статус 401)

```
curl -H 'Authorization: Bearer test123'
http://localhost:3000/api/profile
```

Ответ будет:

{ "user": "Информация о пользователе" } (статус 200)

Порядок подключения middleware

Middleware выполняются строго в том порядке, в котором зарегистрированы через app.use(). Это критически важно:

1) express.json() — должен быть в самом начале, чтобы тело запроса было распарсено для всех маршрутов.

2) Логирование — после парсинга, но до маршрутов, чтобы логировать все запросы.

3) Маршруты — основная логика приложения.

4) Обработка ошибок — в самом конце (о ней подробнее в разделе 5.6).

Пример правильного порядка:

```
app.use(express.json());
app.use(logger);
app.get('/', (req, res) => { res.json({ ok: true }); });
app.use(errorHandler); // обработчик ошибок — всегда последний
```

5.5 Router — модульная организация

По мере роста приложения все роуты в одном файле становятся неуправляемыми. Express.Router позволяет выносить группы роутов в отдельные файлы:

```
// routes/users.js
import { Router } from 'express';
const router = Router();

let users = [];

router.get('/', (req, res) => res.json(users));
router.get('/:id', (req, res) => { /* ... */ });
router.post('/', (req, res) => { /* ... */ });
router.patch('/:id', (req, res) => { /* ... */ });
router.delete('/:id', (req, res) => { /* ... */ });

export default router;
```

В главном файле подключаем роутер с префиксом:

```
// server.js
import express from 'express';
import usersRouter from './routes/users.js';
import productsRouter from './routes/products.js';

const app = express();
app.use(express.json());

// Все роуты из usersRouter будут иметь префикс /api/users
app.use('/api/users', usersRouter);
app.use('/api/products', productsRouter);

app.listen(3000);
```

Теперь router.get("/") в users.js соответствует GET /api/users, а router.get("/:id") — GET /api/users/:id. Это делает код модульным и легко поддерживаемым.

Зачем нужен Router?

Когда приложение растёт, в server.js накапливаются десятки маршрутов для разных сущностей: пользователи, продукты, заказы, авторизация и

т.д. Файл становится огромным и тяжело поддерживаемым. Express.Router решает эту проблему — он позволяет группировать связанные маршруты в отдельные файлы-модули.

Router — это «мини-приложение», которое поддерживает те же методы, что и app (get, post, put, patch, delete, use), но работает изолированно. Затем его подключают к основному приложению через app.use() с указанием префикса.

Выносим маршруты в отдельные файлы

Создайте папку routes/ в корне проекта:

```
mkdir routes
```

Создайте файл routes/users.js:

```
import { Router } from 'express';

const router = Router();

let users = [
    { id: 1, name: 'Алексей', email: 'alex@example.com' },
    { id: 2, name: 'Мария', email: 'maria@example.com' },
];

// GET /api/users — получить всех пользователей
router.get('/', (req, res) => {
    res.json(users);
});

// GET /api/users/:id — получить одного по ID
router.get('/:id', (req, res) => {
    const user = users.find(u => u.id === +req.params.id);
    if (!user) return res.status(404).json({ error: 'Не найден' });
    res.json(user);
});

// POST /api/users — создать нового
router.post('/', (req, res) => {
    const { name, email } = req.body;
```

```
    const user = { id: Date.now(), name, email };
    users.push(user);
    res.status(201).json(user);
});

export default router;
```

Обратите внимание: внутри Router пути указываются относительно. Мы пишем '/', а не '/api/users' — префикс добавится при подключении.

Аналогично создайте файл routes/products.js:

```
import { Router } from 'express';

const router = Router();

let products = [{ id: 1, name: 'Телефон', price: 500 }];

router.get('/', (req, res) => res.json(products));

export default router;
```

Обновите server.js — подключите роутеры с префиксами:

```
import express from 'express';
import usersRouter from './routes/users.js';
import productsRouter from './routes/products.js';

const app = express();
app.use(express.json());

app.use('/api/users', usersRouter);
app.use('/api/products', productsRouter);

app.listen(3000, () => {
    console.log('Сервер: http://localhost:3000');
});
```

Запустите сервер и проверьте:

```
node server.js
```

Проверьте маршруты через curl:

```
curl http://localhost:3000/api/users
curl http://localhost:3000/api/users/1
curl http://localhost:3000/api/products
```

Маршрут router.get('/') в users.js теперь соответствует GET /api/users, а router.get('/:id') — GET /api/users/:id. Префикс /api/users задаётся один раз при подключении в server.js.

Структура проекта после рефакторинга:

```
my-express-app/
  package.json
  server.js
  routes/
    users.js
    products.js
```

Такая структура легко масштабируется: для каждой новой сущности создаётся отдельный файл в routes/, и server.js остаётся чистым и коротким.

Router также поддерживает middleware. Можно применить middleware ко всем маршрутам внутри роутера:

```
// routes/admin.js
import { Router } from 'express';
const router = Router();

// Middleware только для маршрутов этого роутера
router.use((req, res, next) => {
    if (!req.headers['x-admin-key']) {
        return res.status(403).json({ error: 'Доступ запрещён' });
    }
    next();
});

router.get('/stats', (req, res) => res.json({ users: 100 }));

export default router;
```

В этом примере middleware с проверкой x-admin-key сработает только для маршрутов внутри admin-роутера, а не для всего приложения.

5.6 Обработка ошибок

В Express есть специальный тип middleware для обработки ошибок — он принимает 4 аргумента (err, req, res, next). Express определяет его как обработчик ошибок именно по количеству аргументов.

```
// Обработчик ошибок (ЧЕТЫРЕ аргумента!)
const errorHandler = (err, req, res, next) => {
    console.error(err.stack);
    res.status(err.status || 500).json({
        error: err.message || 'Внутренняя ошибка сервера',
    });
};
```

Чтобы передать ошибку в обработчик, вызовите next(err) в любом роуте или middleware:

```
app.get('/api/users/:id', (req, res, next) => {
    try {
        const user = users.find(u => u.id === +req.params.id);
        if (!user) {
            const err = new Error('Пользователь не найден');
            err.status = 404;
            throw err;
        }
        res.json(user);
    } catch (err) {
        next(err); // Express передаст ошибку в errorHandler
    }
});
```

Важно: обработчик ошибок и обработчик 404 регистрируются ПОСЛЕ всех роутов:

```
// Сначала все роуты
app.use('/api/users', usersRouter);


// Потом 404 для несуществующих маршрутов
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});


// В самом конце — обработчик ошибок
app.use(errorHandler);
```

Зачем нужен централизованный обработчик ошибок?

Без него каждый маршрут обрабатывает ошибки по-своему: где-то отправляется JSON, где-то строка, где-то ошибка вообще не ловится. Централизованный обработчик решает эту проблему — все ошибки проходят через одну функцию и отдаются клиенту в одинаковом формате.

Как Express определяет обработчик ошибок?

По количеству аргументов: если middleware-функция принимает ровно 4 аргумента (err, req, res, next), Express считает её обработчиком ошибок. Все четыре аргумента обязательны, даже если next не используется — иначе Express не распознает функцию как error handler.

Добавляем обработку ошибок в приложение

Откройте файл server.js.

Создайте вспомогательный класс для ошибок API. Добавьте в начало файла (или вынесите в отдельный файл utils/ApiError.js):

```
class ApiError extends Error {
    constructor(status, message) {
        super(message);
        this.status = status;
    }
}
```

Это позволит бросать ошибки с нужным HTTP-статусом: throw new ApiError(404, "Не найден").

Создайте обработчик ошибок. Добавьте его в конец файла, ПОСЛЕ всех маршрутов:

```
const errorHandler = (err, req, res, next) => {
    console.error(`[ERROR] ${err.message}`);
    console.error(err.stack);

    const status = err.status || 500;
    const message = err.status ? err.message : 'Внутренняя ошибка
сервера';
```

```
    res.status(status).json({ error: message });
};
```

Обратите внимание: если у ошибки нет свойства status (неожиданная ошибка), мы отдаём статус 500 и общее сообщение. Это важно для безопасности — клиент не должен видеть детали внутренних ошибок.

Добавьте обработчик 404 для несуществующих маршрутов. Он должен стоять ПОСЛЕ всех роутов, но ПЕРЕД errorHandler:

```
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});
```

Подключите errorHandler в самом конце:

```
app.use(errorHandler);
```

Используйте throw и next(err) в маршрутах:

```
app.get('/api/users/:id', (req, res, next) => {
    try {
        const user = users.find(u => u.id === +req.params.id);
        if (!user) {
            throw new ApiError(404, 'Пользователь не найден');
        }
        res.json(user);
    } catch (err) {
        next(err);
    }
});
```

Когда вызывается next(err), Express пропускает все обычные middleware и маршруты и переходит сразу к errorHandler.

Запустите сервер и проверьте:

```
node server.js
```

Проверьте через curl:

```
curl http://localhost:3000/api/users/999
```

Ответ:

{ "error": "Пользователь не найден" } (статус 404)

```
curl http://localhost:3000/nonexistent
```

Ответ:

{ "error": "Маршрут не найден" } (статус 404)

Итоговый порядок в server.js

Правильный порядок регистрации middleware и маршрутов критически важен:

1) express.json() — парсинг тела запроса.

2) Пользовательские middleware (логгер, авторизация и т.д.).

3) Маршруты (app.use("/api/users", usersRouter) и т.д.).

4) Обработчик 404 — ловит запросы, которые не совпали ни с одним маршрутом.

5) Обработчик ошибок (errorHandler) — ловит все ошибки, переданные через next(err).

Пример итоговой структуры server.js:

```
import express from 'express';
import usersRouter from './routes/users.js';
import productsRouter from './routes/products.js';

const app = express();

// 1) Парсинг
app.use(express.json());

// 2) Логирование
app.use((req, res, next) => {
    console.log(`${req.method} ${req.url}`);
```

```
    next();
});

// 3) Маршруты
app.use('/api/users', usersRouter);
app.use('/api/products', productsRouter);

// 4) 404
app.use((req, res) => {
    res.status(404).json({ error: 'Маршрут не найден' });
});

// 5) Обработка ошибок
app.use((err, req, res, next) => {
    console.error(err.stack);
    const status = err.status || 500;
    res.status(status).json({ error: err.message || 'Внутренняя ошибка
сервера' });
});

app.listen(3000, () => {
    console.log('Сервер: http://localhost:3000');
});
```

5.7 Финалочка

Express.js — это один из самых популярных фреймворков для серверной разработки на платформе Node.js. Он используется для создания веб-серверов, REST API, микросервисов и backend-части современных веб-приложений.

Главное преимущество Express заключается в его простоте и минимализме. Фреймворк не навязывает строгую архитектуру и предоставляет разработчику только базовые инструменты для работы с HTTP-запросами, маршрутизацией и middleware. Благодаря этому разработчики могут быстро создавать серверные приложения и гибко организовывать структуру проекта.

Express также обладает огромной экосистемой пакетов и расширений. Через менеджер пакетов npm можно подключить тысячи готовых модулей для обработки данных, аутентификации, логирования, работы с файлами, безопасности и других задач. Это значительно ускоряет разработку и позволяет не писать многие вещи с нуля.

Фреймворк широко используется для разработки backend-части веб-приложений, мобильных сервисов и API, которые взаимодействуют с клиентскими приложениями. Он хорошо подходит для создания микросервисной архитектуры и серверов, обслуживающих современные одностраничные приложения.

Express стал фактическим стандартом серверной разработки в экосистеме Node.js. Многие более сложные фреймворки и платформы для разработки серверных приложений построены поверх него или используют его принципы. Поэтому понимание Express помогает легче изучать другие инструменты Node.js-разработки.

Важно отметить, что крупнейшие социальные сети и технологические платформы, такие как Facebook, Instagram, TikTok и VK, как правило, не используют Express в качестве основного backend-решения. Такие компании работают с огромными нагрузками и часто применяют собственные технологии или более низкоуровневые языки и платформы.

Тем не менее Express активно используется тысячами компаний, стартапов и разработчиков по всему миру для создания веб-сервисов, API и серверной логики приложений. Благодаря простоте, гибкости и широкой поддержке сообщества он остаётся одним из ключевых инструментов в современной JavaScript-разработке.

Практика

Задание 5.1: REST API для задач (Todo):

- GET /api/todos — список задач (с фильтрацией по ?completed=true)
- GET /api/todos/:id — одна задача
- POST /api/todos — создать задачу { title, completed }
- PATCH /api/todos/:id — обновить
- DELETE /api/todos/:id — удалить
- Middleware: логгер с временем выполнения, обработка ошибок
- Вынесите роуты в routes/todos.js, используйте Router
- Добавьте express.static для папки public/ с простым HTML-интерфейсом